

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

EMBEDDED SYSTEMS

VI SEMESTER

UNIT – I

Introduction to Embedded Systems - Processor in Embedded System – Other Hardware Units in the Embedded System - Software Embedded into a System - ARM Architecture: ARM Design Philosophy - Registers - Program Status Register - Instruction Pipeline - Interrupts and Vector Table - Architecture Revision - ARM Processor Families.

UNIT – II

ARM Programming - Instruction Set - Data Processing Instructions - Addressing Modes - Branch, Load, Store Instructions - PSR Instructions - Conditional Instructions.

UNIT – III

Thumb Instruction Set - Register Usage - Other Branch Instructions - Data Processing Instructions - Single-Register and Multi Register Load-Store Instructions - Stack - Software Interrupt Instructions

UNIT – IV

ARM Programming using C: Simple C Programs using Function Calls – Pointers – Structures - Integer and Floating Point Arithmetic - Assembly Code using Instruction Scheduling – Register Allocation - Conditional Execution and Loops.

UNIT – V

Real Time Operating Systems: Brief History of OS - Defining RTOS - The Scheduler - Objects – Services - Characteristics of RTOS - Defining a Task - Tasks States and Scheduling - Task Operations – Structure – Synchronization - Communication and Concurrency. Defining Semaphores - Operations and Use - Defining Message Queue - States – Content – Storage - Operations and Use

Text Books:

1. Shibu K.V, Introduction to Embedded Systems, First Edition, McGraw Hill, 2009.
2. Andrew N. Sloss, Dominic Symes, Chris Wright, ARM Systems Developer's Guides- Designing & Optimizing System Software, Elsevier, 2008.
3. Qing Li , Real Time Concepts for Embedded Systems, Elsevier, 2011

Reference Books:

1. Santanu Chattopadhyay, "Embedded System Design", Second Edition, PHI, 2013.
2. Andrew N Sloss, D. Symes and C. Wright, "ARM System Developers Guide", Morgan Kaufmann / Elsevier, 2006.
3. Wayne Wolf, "Computer as Components: Principles of Embedded Computer System Design", Elsevier, 2006

Websites:

1. <http://infocenter.arm.com/help/index.jsp?topic=/com.arm.doc.dai0211a/index.htm>
2. <http://www.arm.com/products/processors/classic/arm7/index.php>
3. <http://infocenter.arm.com/help/index.jsp?topic=/com.arm.doc.dai0211a/index.htm>

UNIT-I

Introduction to Embedded Systems - Processor in Embedded System – Other Hardware Units in the Embedded System - Software Embedded into a System - ARM Architecture: ARM Design Philosophy - Registers - Program Status Register - Instruction Pipeline - Interrupts and Vector Table - Architecture Revision - ARM Processor Families.

INTRODUCTION TO EMBEDDED SYSTEMS:

- An embedded system is a microprocessor-based system that is built to control a function or range of functions and is not designed to be programmed by the end user in the same way that a PC is.
- An embedded system is designed to perform one particular task albeit with choices and different options.

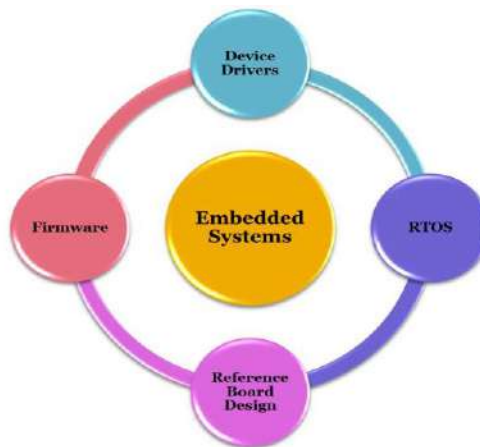


Figure: The design of Embedded System

DEFINITION:

Embedded system:

An embedded system is a special-purpose computer system designed to perform a dedicated function. Unlike a general-purpose computer, such as a personal computer, an embedded system performs one or a few pre-defined tasks, usually with very specific requirements, and often includes task-specific hardware and mechanical parts not usually found in a general-purpose computer.

Embedded computer System:

Any device that includes a programmable computer but is not itself intended to be a general-purpose computer is called an embedded computer system.

Embedding Computers:

Computers have been embedded into applications since the earliest days of computing. Eg) Whirlwind, a computer designed at MIT in the late 1940s and early 1950s. Whirlwind was also the first computer

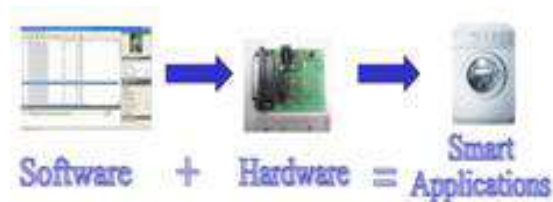
designed to support real-time operation and was originally conceived as a mechanism for controlling an aircraft simulator.

Computer hardware that can be used:

- A Microprocessor
- A Large Memory (Ram, ROM and caches)
- Input Units (Keyboard, Mouse, Scanner, etc)
- Output Units (Monitor, Printer, etc)
- Networking Units (Ethernet Card, Drivers, etc)
- I/O Units (Modem, Fax cum modem, etc)

➤ Embedded system is a

- Microcontroller based
- Software driven
- Reliable
- Real time control system
- Autonomous or Human interactive
- Operating on diverse physical variable
- In diverse environments



➤ Embedded system is Hardware with Software embedded in it, for a dedicated application.

Microprocessor:

A microprocessor is a single-chip CPU. The first microprocessor, the Intel 4004, was designed for an embedded application, namely, a calculator. Microprocessors come in many different levels of sophistication; they are usually classified by their word size. Microprocessors execute programs very efficiently.

- An **8-bit microcontroller** is designed for low-cost applications and includes on-board memory and I/O devices;
- A **16-bit microcontroller** is often used for more sophisticated applications that may require either longer word lengths or off-chip I/O and memory;
- A **32-bit RISC** microprocessor offers very high performance for computation-intensive applications.

Why use microprocessors?

- Microprocessors are a very efficient way to implement digital systems.
- Microprocessors make it easier to design families of products that can be built to provide various feature sets at different price points and can be extended to provide new features to keep up with rapidly changing markets.

Characteristics of Embedded Computing Applications

Embedded computing is in many ways much more demanding than the sort of programs that may have been written for PCs or workstations. On the one hand, embedded computing systems have to provide sophisticated functionality:

■ **Complex algorithms:** The operations performed by the microprocessor may be very sophisticated. For example, the microprocessor that controls an automobile engine must perform complicated filtering functions to optimize the performance of the car while minimizing pollution and fuel utilization.

■ **User interface:** Microprocessors are frequently used to control complex user interfaces that may include multiple menus and many options. The moving maps in Global Positioning System (GPS) navigation are good examples of sophisticated user interfaces.

Deadlines involves:

■ **Real time:** Many embedded computing systems have to perform in real time if the data is not ready by a certain deadline, the system breaks. In some cases, failure to meet a deadline is unsafe and can even endanger lives. In other cases, missing a deadline does not create safety problems but does create unhappy customers—missed deadlines in printers, can result in scrambled pages.

■ **Multirate:** Many embedded computing systems have several real-time activities going on at the same time. They may simultaneously control some operations that run at slow rates and others that run at high rates. Multimedia applications are prime examples of multirate behavior. The audio and video portions of a multimedia stream run at very different rates, but they must remain closely synchronized.

➤ Costs of various sorts are also very important:

■ **Manufacturing cost:** The total cost of building the system is very important in many cases. Manufacturing cost is determined by many factors, including the type of microprocessor used, the amount of memory required, and the types of I/O devices.

■ **Power and energy:** Power consumption directly affects the cost of the hardware, since a larger power supply may be necessary. Energy consumption affects battery life, which is important in many applications, as well as heat consumption, which can be important even in desktop applications.

PROCESSOR IN EMBEDDED SYSTEM:

PROCESSOR

- The main part of an embedded system is its processor. This can be a generic microprocessor or a microcontroller. The processor is programmed to perform the specific tasks for which the embedded system has been designed.
- The main criteria for the processor is: can it provide the processing power needed to perform the tasks within the system.
- Program Flow and data path Control Unit (CU) —includes a fetch unit for fetching instructions from the memory.
- Execution Unit (EU) —includes circuits for arithmetic and logical unit (ALU), and for instructions for a program control task, say, data transfer instructions, halt, interrupt, or jump to another set of instructions or call to another routine or sleep or reset.

SYSTEM DESIGNER CONSIDERATIONS:

- Processor Instructions in the Instruction set
- Processor ability to solve the complex algorithms used in meeting the deadlines for their processing.
- Maximum bits in operand (8 or 16 or 32) in a single arithmetic or logical operation.
- Internal and External bus-widths in the data-path
- Clock frequency in MHz and processing speed – Million Instructions Per Second (MIPS) or Million Floating Point Instructions Per Second (MFLOPS) or Dhrystone– an alternate metric for measuring processing performance.

THE LIST OF PROCESSOR:

- General purpose microprocessor For example, Intel 80x86, Sparc, or Motorola 68HCxxx. Embedded general purpose processor Fast context switching features, use of on-chip Compilers, for example, Intel® XScale™ Applications Personal Internet Client Architecture-based PDAs, cell phones and other wireless devices,
- Application Specific Instruction Set Processor (ASIP)
- (a)Microcontroller — Intel, Motorola, Hitachi, TI, Philips and ARM: an Intel® — MCS51, Philips® 51XA, 51MX, or Motorola — 68HC11, 68HC12, 68HC16
- (b)Application Specific Instruction-Set Processor (ASIP): DSP or Media processor or IO processor or Network processor or A domain specific processor
- DSP: Typically a Texas Instruments- C28x Series, C54xx or C64xx or Analog Devices SHARC or TigerSHARC, Motorola 5600xx.
- Media processor: TI DSP TMS320DM310 or Trimedia Phillips Media Processor 1x00 series for Processing Streaming and Data Networks and Image, Video and Speech: PNX 1300, PNX 1500 (2002)
- GPP or ASIP core (s): GPP or ASIP Integrated into either an Application Specific Integrated Circuit (ASIC), or a Very Large Scale Integrated Circuit (VLSI) circuit or a FPGA core integrated with processor unit(s) in a VLSI (ASIC) chip.
- .Application Specific System Processor (ASSP): Typically a set top box processor or mpeg video-processor or network application processor or mobile application processor
- Single purpose processor or Single purpose processor Application Specific Instruction processor:
 - Floating point Coprocessor
 - CCD Pixel coprocessor and image codec in digital camera

- Graphic processor
- Speech processor
- Adaptive filtering processor
- Encryption engine
- Decryption engine
- Communication protocol stack processor
- Java accelerator

Use of Accelerator Cores:

- Examples :
 - Java Accelerator Nazonin Communications Java codes run 15 to 60 Times fast, Video Accelerator for fast Video processing Multi core processors or Multi core processors or multiprocessor system using multiprocessor system using GPPs:
- Examples:
 - Multiprocessor system for Real time performance in a video-conference system
 - Embedded firewall cum router
 - High-end cell phone

OTHER HARDWARE UNITS IN THE EMBEDDED SYSTEM:

- Typical Embedded System Hardware Unit:
- Program Flow and data path Control Unit (CU): comprises a fetch unit for fetching instructions through the memory.

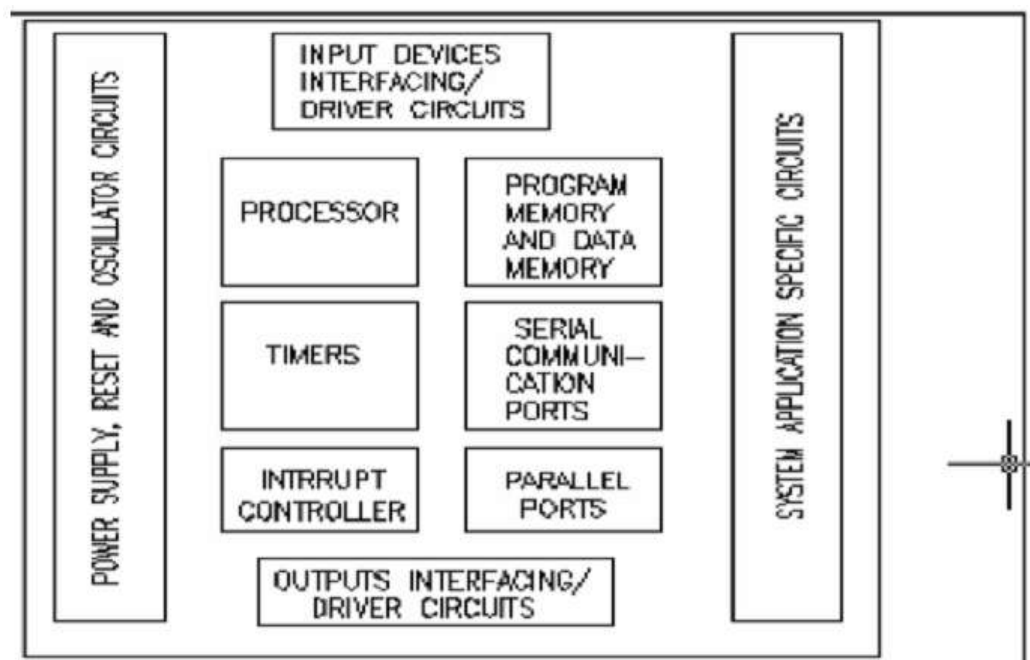


Fig: Embedded System Hardware Unit

MEMORY

- Electronic memory is an important part of embedded systems. This memory is of essentially three types: RAM, or random access memory, ROM, or read-only memory, and cache.
- The RAM is where program components are temporarily stored during execution. The ROM contains the basic input-output routines that are needed by the system at startup.
- The cache is used by the processor as a temporary storage during processing and data transfer.

MEMORY CONTROLLERS:

- Memory controllers connect different types of memory to the processor bus. On power-up a memory controller is configured in hardware to allow certain memory devices to be active.
- These memory devices allow the initialization code to be executed. Some memory devices must be set up by software; for example, when using DRAM, first have to set up the memory timings and refresh rate before it can be accessed.

SYSTEM CLOCK

- The system clock is a very important part of an embedded system since all processes in an embedded system run on clock cycles and require precise timing information.
- This clock generally consists of an oscillator and some associated circuitry.

PERIPHERALS

- The peripherals interface an embedded system with other components. The peripheral devices are provided on the embedded system boards for easy integration.
- Typical peripherals include serial port, parallel port, network port, keyboard and mouse ports, memory drive port and monitor port.
- Some specialized embedded systems also have other ports such as CAN-bus port.

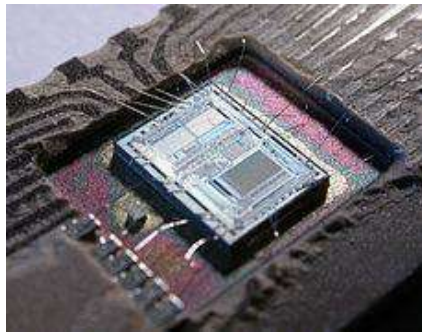
INTERRUPT CONTROLLERS:

- When a peripheral or device requires attention, it raises an interrupt to the processor.
- An interrupt controller provides a programmable governing policy that allows software to determine which peripheral or device can interrupt the processor at any specific time by setting the appropriate bits in the interrupt controller registers.

➤ PROCESSOR:

MICROCONTROLLER

- A **microcontroller** (sometimes abbreviated **μC**, **uC** or **MCU**) is a small computer on a single integrated circuit containing a processor core, memory, and programmable input/output peripherals.
- Program memory in the form of NOR flash or OTP ROM is also often included on chip, as well as a typically small amount of RAM.
- Microcontrollers are designed for embedded applications, in contrast to the microprocessors used in personal computers or other general purpose applications.



- Microcontrollers are used in automatically controlled products and devices, such as automobile engine control systems, implantable medical devices, remote controls, office machines, appliances, power tools, and toys.
- By reducing the size and cost compared to a design that uses a separate microprocessor, memory, and input/output devices, microcontrollers make it economical to digitally control even more devices and processes.
- Mixed signal microcontrollers are common, integrating analog components needed to control non-digital electronic systems

VARIOUS MICROCONTROLLERS

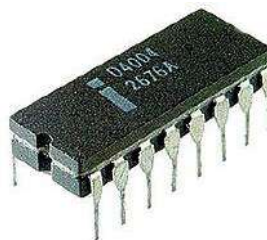
INTEL 8031, 8032, 8051, 8052, 8751, 8752

PIC 8bit PIC 16, PIC 18
 16bit DSPIC 33/PIC 24
 PIC 16C7X

MOTOROLO MC 68HC11

MICROPROCESSOR

- A **microprocessor** incorporates the functions of a computer's central processing unit (CPU) on a single integrated circuit (IC, or microchip).
- It is a multipurpose, programmable, and clock-driven, register based electronic device that accepts binary data as input, processes it according to instructions stored in its memory, and provides results as output.



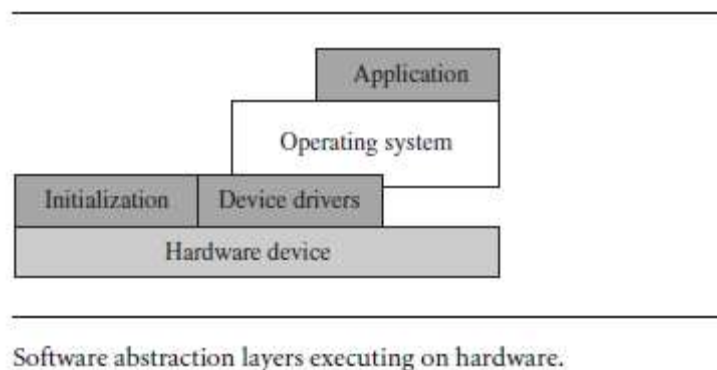
- The first microprocessors emerged in the early 1970s and were used for electronic calculators, using binary-coded decimal (BCD) arithmetic on 4-bit words.
- Other embedded uses of 4-bit and 8-bit microprocessors, such as terminals, printers, various kinds of automation etc., followed soon after.
- Affordable 8-bit microprocessors with 16-bit addressing also led to the first general-purpose microcomputers from the mid-1970s on.
- Example: Intel 4004, the first general-purpose, commercial microprocessor.

VARIOUS MICROPROCESSORS

INTEL	Zilog
4004, 4040	Z80, Z180, 1280
8080, 8085	Z8, eZ8
8086, 8088	and others
80186, 80188	
80286, 80386	
X86-64	

SOFTWARE EMBEDDED INTO A SYSTEM:

- The software components within an embedded system often encompass the technology that adds value to the system and defines what it does and how well it does it.
- The software can consist of several different components:
 - Initialization and configuration
 - Operating system or run-time environment
 - The applications software itself
 - Error handling
 - Debug and maintenance support.
- An embedded system needs software to drive it.
- The figure shows four typical software components required to control an embedded device.
- Each software component in the stack uses a higher level of abstraction to separate the code from the hardware device.
- The initialization code is the first code executed on the board and is specific to a particular target or group of targets.
- It sets up the minimum parts of the board before handing control over to the operating system.



- The operating system provides an infrastructure to control applications and manage hardware system resources.
- Many embedded systems do not require a full operating system but merely a simple task scheduler that is either event or poll driven.
- The device drivers are the third component shown in Figure .
- They provide a consistent software interface to the peripherals on the hardware device.
- Finally, an application performs one of the tasks required for a device. For example, a mobile phone might have a diary application.
- There may be multiple applications running on the same device, controlled by the operating system.
- The software components can run from ROM or RAM. ROM code that is fixed on the device (for example, the initialization code) is called firmware.

ARM ARCHITECTURE:

INTRODUCTION:

- The ARM processor core is a key component of many successful 32-bit embedded systems.
- ARM's designers have come a long way from the first ARM1 prototype in 1985. Over one billion ARM processors had been shipped worldwide by the end of 2001.
- The ARM Company bases their success on a simple and powerful original design, which continues to improve today through constant technical innovation.
- In fact, the ARM core is not a single core, but a whole family of designs sharing similar design principles and a common instruction set.

ARM DESIGN PHILOSOPHY

- There are a number of physical features that have driven the ARM processor design.
- First, portable embedded systems require some form of battery power.
- The ARM processor has been specifically designed to be small to reduce power consumption and extend battery operation essential for applications such as mobile phones and personal digital assistants (PDAs).
- High code density is another major requirement since embedded systems have limited memory due to cost and/or physical size restrictions.
- High code density is useful for applications that have limited on-board memory, such as mobile phones and mass storage devices.
- In addition, embedded systems are price sensitive and use slow and low-cost memory devices.
- For high-volume applications like digital cameras, every cent has to be accounted for in the design.
- The ability to use low-cost memory devices produces substantial savings.
- Another important requirement is to reduce the area of the die taken up by the embedded processor.
- For a single-chip solution, the smaller the area used by the embedded processor, the more available space for specialized peripherals.
- This in turn reduces the cost of the design and manufacturing since fewer discrete chips are required for the end product.

- ARM has incorporated hardware debug technology within the processor so that software engineers can view what is happening while the processor is executing code.
- With greater visibility, software engineers can resolve issues faster, which have a direct effect on the time to market and reduce overall development costs.

REGISTERS:

- General-purpose registers hold either data or an address. They are identified with the letter *r* prefixed to the register number. For example, register 4 is given the label *r4*.
- The figure shows the active registers available in user mode—a protected mode normally used when executing applications.
- The processor can operate in seven different modes. All the registers shown are 32 bits in size.
- There are up to 18 active registers: 16 data registers and 2 processor status registers. The data registers are visible to the programmer as *r0* to *r15*.
- The ARM processor has three registers assigned to a particular task or special function: *r13*, *r14*, and *r15*. They are frequently given different labels to differentiate them from the other register.
- In Figure, the shaded registers identify the assigned special-purpose registers: Register *r13* is traditionally used as the stack pointer (*sp*) and stores the head of the stack in the current processor mode. Register *r14* is called the link register (*lr*) and is where the core puts the return address whenever it calls a subroutine.
- Register *r15* is the program counter (*pc*) and contains the address of the next instruction to be fetched by the processor.

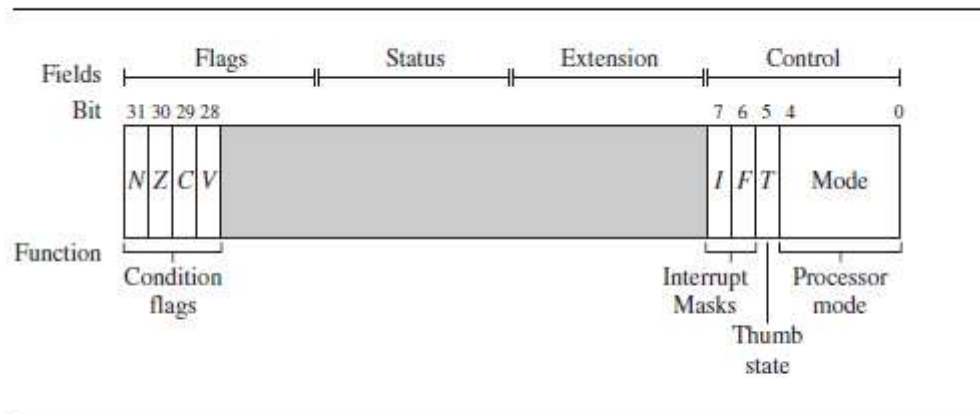
<i>r0</i>
<i>r1</i>
<i>r2</i>
<i>r3</i>
<i>r4</i>
<i>r5</i>
<i>r6</i>
<i>r7</i>
<i>r8</i>
<i>r9</i>
<i>r10</i>
<i>r11</i>
<i>r12</i>
<i>r13 sp</i>
<i>r14 lr</i>
<i>r15 pc</i>
<i>cpsr</i>
-

Registers available in *user* mode.

- Depending upon the context, registers *r13* and *r14* can also be used as general-purpose registers, which can be particularly useful since these registers are banked during a processor mode change.
- However, it is dangerous to use *r13* as a general register when the processor is running any form of operating system because operating systems often assume that *r13* always points to a valid stack frame.

- In ARM state the registers r0 to r13 are orthogonal—any instruction that can apply to r0 it can equally well apply to any of the other registers.
- However, there are instructions that treat r14 and r15 in a special way.
- In addition to the 16 data registers, there are two program status registers: cpsr and spsr (the current and saved program status registers, respectively).
- The register file contains all the registers available to a programmer. Which registers are visible to the programmer depend upon the current mode of the processor.

PROGRAM STATUS REGISTER:



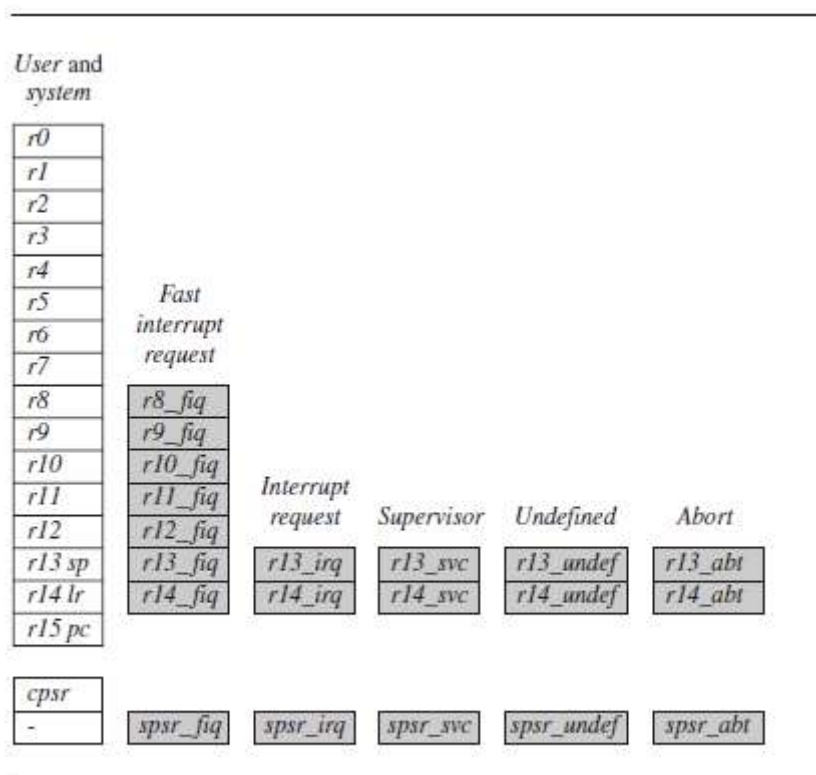
A generic program status register (*psr*).

- The ARM core uses the cpsr to monitor and control internal operations. The cpsr is a dedicated 32-bit register and resides in the register file.
- The figure shows the basic layout of a generic program status register. Note that the shaded parts are reserved for future expansion.
- The cpsr is divided into four fields, each 8 bits wide: flags, status, extension, and control.
- In current designs the extension and status fields are reserved for future use. The control field contains the processor mode, state, and interrupts mask bits. The flags field contains the condition flags.
- Some ARM processor cores have extra bits allocated. For example, the J bit, which can be found in the flags field, is only available on Jazelle-enabled processors, which execute 8-bit instructions.

PROCESSOR MODES:

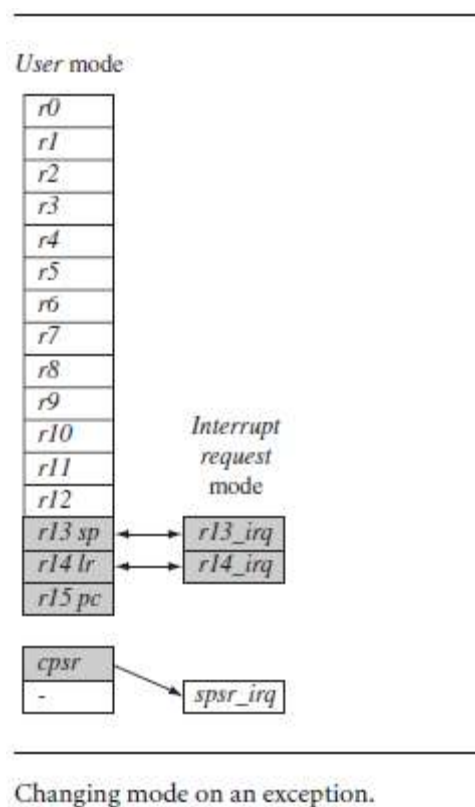
- The processor mode determines which registers are active and the access rights to the cpsr register itself.
- Each processor mode is either privileged or nonprivileged: A privileged mode allows full read-write access to the cpsr.
- Conversely, a nonprivileged mode only allows read access to the control field in the cpsr but still allows read-write access to the condition flags.
- There are seven processor modes in total: six privileged modes (abort, fast interrupt request, interrupt request, supervisor, system, and undefined) and one non privileged mode (user).
- The processor enters abort mode when there is a failed attempt to access memory. Fast interrupt request and interrupt request modes correspond to the two interrupt levels available on the ARM processor.
- Supervisor mode is the mode that the processor is in after reset and is generally the mode that an operating system kernel operates in. System mode is a special version of user mode that allows full read-write access to the cpsr.
- Undefined mode is used when the processor encounters an instruction that is undefined or not supported by the implementation. User mode is used for programs and applications.

BANKED REGISTERS:



Complete ARM register set.

- Figure shows all 37 registers in the register file. Of those, 20 registers are hidden from a program at different times. These registers are called banked registers and are identified by the shading in the diagram.
- They are available only when the processor is in a particular mode; for example, abort mode has banked registers `r13_abt`, `r14_abt` and `spsr_abt`.
- Banked registers of a particular mode are denoted by an underline character post-fixed to the mode mnemonic or `_mode`.
- The processor mode can be changed by a program that writes directly to the `cpsr` (the processor core has to be in privileged mode) or by hardware when the core responds to an exception or interrupt.
- The following exceptions and interrupts cause a mode change: reset, interrupt request, fast interrupt request, software interrupt, data abort, prefetch abort, and undefined instruction.
- Exceptions and interrupts suspend the normal execution of sequential instructions and jump to a specific location. Figure illustrates what happens when an interrupt forces a mode change.



- The figure shows the core changing from user mode to interrupt request mode, which happens when an interrupt request occurs due to an external device raising an interrupt to the processor core. This change causes user registers `r13` and `r14` to be banked. The user registers are replaced with registers `r13_irq` and `r14_irq`, respectively.
- Note `r14_irq` contains the return address and `r13_irq` contains the stack pointer for interrupt request mode.

STATE AND INSTRUCTION SETS:

- The state of the core determines which instruction set is being executed. There are three instruction sets: ARM, Thumb, and Jazelle. The ARM instruction set is only active when the processor is in ARM state. Similarly the Thumb instruction set is only active when the processor is in Thumb state.
- Once in Thumb state the processor is executing purely Thumb 16-bit instructions. IT cannot intermingle sequential ARM, Thumb, and Jazelle instructions. The Jazelle J and Thumb T bits in the cpsr reflect the state of the processor. When both J and T bits are 0, the processor is in ARM state and executes ARM instructions.
- This is the case when power is applied to the processor. When the T bit is 1, then the processor is in Thumb state. To change states the core executes a specialized branch instruction.
- The ARM designers introduced a third instruction set called Jazelle. Jazelle executes 8-bit instructions and is a hybrid mix of software and hardware designed to speed up the execution of Java byte codes.
- To execute Java byte codes, it requires the Jazelle technology plus a specially modified version of the Java virtual machine. It is important to note that the hardware portion of Jazelle only supports a subset of the Java byte codes; the rest are emulated in software.

INTERRUPT MASKS:

- Interrupt masks are used to stop specific interrupt requests from interrupting the processor.
- There are two interrupt request levels available on the ARM processor core—interrupt request (IRQ) and fast interrupt request (FIQ).
- The cpsr has two interrupt mask bits, 7 and 6 (or I and F), which control the masking of IRQ and FIQ, respectively. The I bit masks IRQ when set to binary 1, and similarly the F bit masks FIQ when set to binary 1.

CONDITION FLAGS:

- Condition flags are updated by comparisons and the result of ALU operations that specify the S instruction suffix.
- For example, if a SUBS subtract instruction results in a register value of zero, then the Z flag in the cpsr is set. This particular subtract instruction specifically updates the cpsr.

Condition flags.

Flag	Flag name	Set when
Q	Saturation	the result causes an overflow and/or saturation
V	oVerflow	the result causes a signed overflow
C	Carry	the result causes an unsigned carry
Z	Zero	the result is zero, frequently used to indicate equality
N	Negative	bit 31 of the result is a binary 1

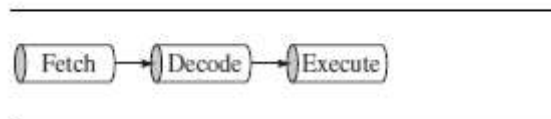
CONDITIONAL EXECUTION:

- Conditional execution controls whether or not the core will execute an instruction.
- Most instructions have a condition attribute that determines if the core will execute it based on the setting of the condition flags. Prior to execution, the processor compares the condition attribute with the condition flags in the cpsr.
- If they match, then the instruction is executed; otherwise the instruction is ignored.

- The condition attribute is postfixed to the instruction mnemonic, which is encoded into the instruction.

INSTRUCTION PIPELINE:

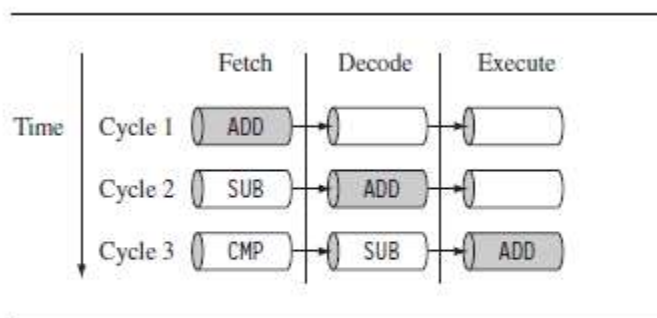
- A pipeline is the mechanism a RISC processor uses to execute instructions.
- Using a pipeline speeds up execution by fetching the next instruction while other instructions are being decoded and executed.



ARM7 Three-stage pipeline.

Figure : shows a three-stage pipeline

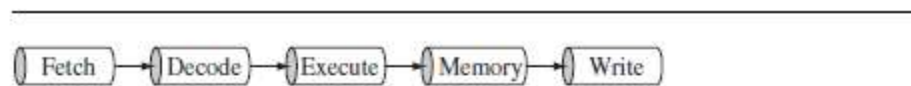
- Fetch loads an instruction from memory.
- Decode identifies the instruction to be executed.
- Execute processes the instruction and writes the result back to a register.



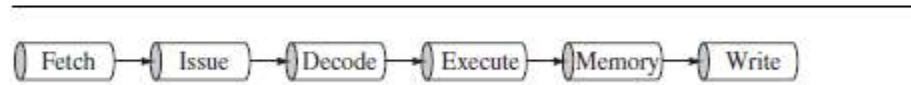
Pipelined instruction sequence.

- The figure shows a sequence of three instructions being fetched, decoded, and executed by the processor.
- Each instruction takes a single cycle to complete after the pipeline is filled.
- The three instructions are placed into the pipeline sequentially.
- In the first cycle the core fetches the ADD instruction from memory.
- In the second cycle the core fetches the SUB instruction and decodes the ADD instruction.
- In the third cycle, both the SUB and ADD instructions are moved along the pipeline.
- The ADD instruction is executed, the SUB instruction is decoded, and the CMP instruction is fetched.
- This procedure is called filling the pipeline. The pipeline allows the core to execute an instruction every cycle.
- As the pipeline length increases, the amount of work done at each stage is reduced, which allows the processor to attain a higher operating frequency.
- This in turn increases the performance.
- The system latency also increases because it takes more cycles to fill the pipeline before the core can execute an instruction.

- The increased pipeline length also means there can be data dependency between certain stages.



ARM9 five-stage pipeline.

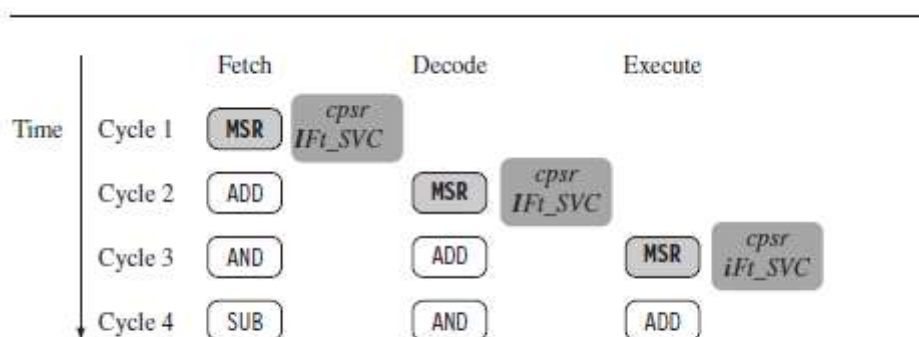


ARM10 six-stage pipeline.

- The pipeline design for each ARMfamily differs. For example, The ARM9 core increases the pipeline length to five stages, as shown in Figure.
- The ARM10 increases the pipeline length still further by adding a sixth stage, as shown in Figure.
- Even though the ARM9 and ARM10 pipelines are different, they still use the same pipeline executing characteristics as an ARM7.
- Code written for the ARM7 will execute on an ARM9 or ARM10.

PIPELINE EXECUTING CHARACTERISTICS:

- The ARM pipeline has not processed an instruction until it passes completely through the execute stage.
- For example, an ARM7 pipeline (with three stages) has executed an instruction only when the fourth instruction is fetched.



ARM instruction sequence.

- The above figure shows an instruction sequence on an ARM7 pipeline.
- The MSR instruction is used to enable IRQ interrupts, which only occurs once the MSR instruction completes the execute stage of the pipeline.
- It clears the I bit in the cpsr to enable the IRQ interrupts.
- Once the ADD instruction enters the execute stage of the pipeline, IRQ interrupts are enabled.

INTERRUPTS AND VECTOR TABLE:

- When an exception or interrupt occurs, the processor sets the pc to a specific memory address.
- The address is within a special address range called the vector table.
- The entries in the vector table are instructions that branch to specific routines designed to handle a particular exception or interrupt.
- The memory map address 0x00000000 is reserved for the vector table, a set of 32-bit words.
- On some processors the vector table can be optionally located at a higher address in memory (starting at the offset 0xffff0000).
- Operating systems such as Linux and Microsoft's embedded products can take advantage of this feature.
- When an exception or interrupt occurs, the processor suspends normal execution and starts loading instructions from the exception vector table (see Table).
- Each vector table entry contains a form of branch instruction pointing to the start of a specific routine:

The vector table.

Exception/interrupt	Shorthand	Address	High address
Reset	RESET	0x00000000	0xffff0000
Undefined instruction	UNDEF	0x00000004	0xffff0004
Software interrupt	SWI	0x00000008	0xffff0008
Prefetch abort	PABT	0x0000000c	0xffff000c
Data abort	DABT	0x00000010	0xffff0010
Reserved	—	0x00000014	0xffff0014
Interrupt request	IRQ	0x00000018	0xffff0018
Fast interrupt request	FIQ	0x0000001c	0xffff001c

- Reset vector is the location of the first instruction executed by the processor when power is applied.

This instruction branches to the initialization code.

- Undefined instruction vector is used when the processor cannot decode an instruction.
- Software interrupt vector is called when it execute a SWI instruction. The SWI instruction is frequently used as the mechanism to invoke an operating system routine.
- Prefetch abort vector occurs when the processor attempts to fetch an instruction from an address without the correct access permissions. The actual abort occurs in the decode stage.
- Data abort vector is similar to a prefetch abort but is raised when an instruction attempts to access data memory without the correct access permissions.
- Interrupt request vector is used by external hardware to interrupt the normal execution flow of the processor. It can only be raised if IRQs are not masked in the cpsr.

- Fast interrupt request vector is similar to the interrupt request but is reserved for hardware requiring faster response times.

ARCHITECTURE REVISION:

- Every ARM processor implementation executes a specific instruction set architecture (ISA), although an ISA revision may have more than one processor implementation.
- The ISA has evolved to keep up with the demands of the embedded market.
- This evolution has been carefully managed by ARM, so that code written to execute on an earlier architecture revision will also execute on a later revision of the architecture.
- To explain the evolution of the architecture, the ARM processor nomenclature is introduced.
- The nomenclature identifies individual processors and provides basic information about the feature set.

Nomenclature:

- ARM uses the nomenclature shown in figure to describe the processor implementations.

ARM{x}{y}{z}{T}{D}{M}{I}{E}{J}{F}{-S}

x—family

y—memory management/protection unit

z—cache

T—Thumb 16-bit decoder

D—JTAG debug

M—fast multiplier

I—EmbeddedICE macrocell

E—enhanced instructions (assumes TDMI)

J—Jazelle

F—vector floating-point unit

S—synthesizable version

ARM nomenclature.

- The letters and numbers after the word “ARM” indicate the features a processor may have.
- In the future the number and letter combinations may change as more feature are added.
- Note the nomenclature does not include the architecture revision information.
- There are a few additional points to make about the ARM nomenclature:
 - All ARM cores after the ARM7TDMI include the TDMI features even though they may not include those letters after the “ARM” label.
 - The processor family is a group of processor implementations that share the same hardware characteristics.

- For example, the ARM7TDMI, ARM740T, and RM720T all share the same family characteristics and belong to the ARM7 family.
- JTAG is described by IEEE 1149.1 Standard Test Access Port and boundary scan architecture. It is a serial protocol used by ARM to send and receive debug information between the processor core and test equipment.
- Embedded ICE macrocell is the debug hardware built into the processor that allows breakpoints and watchpoints to be set.
- Synthesizable means that the processor core is supplied as source code that can be compiled into a form easily used by EDA tools.

ARM PROCESSOR FAMILIES:

- ARM has designed a number of processors that are grouped into different families according to the core they use.
- The families are based on the ARM7, ARM9, ARM10, and ARM11 cores.
- The postfix numbers 7, 9, 10, and 11 indicate different core designs.
- The ascending number equates to an increase in performance and sophistication. ARM8 was developed but was soon superseded.
- Within each ARM family, there are a number of variations of memory management, cache, and TCM processor extensions.
- ARM continues to expand both the number of families available and the different variations within each family.

ARM7 FAMILY:

- The ARM7 core has a Von Neumann-style architecture, where both data and instructions use the same bus.
- The core has a three-stage pipeline and executes the architecture ARMv4T instruction set.
- The ARM7TDMI was the first of a new range of processors introduced in 1995 by ARM.
- It is currently a very popular core and is used in many 32-bit embedded processors. It provides a very good performance-to-power ratio.
- The ARM7TDMI processor core has been licensed by many of the top semiconductor companies around the world and is the first core to include the Thumb instruction set, a fast multiply instruction, and the Embedded ICE debug technology.

CPU core	MMU/MPU	Cache	Jazelle	Thumb	ISA	E ^a
ARM7TDMI	none	none	no	yes	v4T	no
ARM7EJ-S	none	none	yes	yes	v5TEJ	yes
ARM720T	MMU	unified—8K cache	no	yes	v4T	no
ARM920T	MMU	separate—16K /16K <i>D + I</i> cache	no	yes	v4T	no
ARM922T	MMU	separate—8K/8K <i>D + I</i> cache	no	yes	v4T	no
ARM926EJ-S	MMU	separate—cache and TCMs configurable	yes	yes	v5TEJ	yes
ARM940T	MPU	separate—4K/4K <i>D + I</i> cache	no	yes	v4T	no
ARM946E-S	MPU	separate—cache and TCMs configurable	no	yes	v5TE	yes
ARM966E-S	none	separate—TCMs configurable	no	yes	v5TE	yes
ARM1020E	MMU	separate—32K/32K <i>D + I</i> cache	no	yes	v5TE	yes
ARM1022E	MMU	separate—16K/16K <i>D + I</i> cache	no	yes	v5TE	yes
ARM1026EJ-S	MMU and MPU	separate—cache and TCMs configurable	yes	yes	v5TE	yes
ARM1136J-S	MMU	separate—cache and TCMs configurable	yes	yes	v6	yes
ARM1136JF-S	MMU	separate—cache and TCMs configurable	yes	yes	v6	yes

^a E extension provides enhanced multiply instructions and saturation.

Table:ARM processor variants

ARM9 FAMILY.

- The ARM9 family was announced in 1997. Because of its five-stage pipeline, the ARM9 processor can run at higher clock frequencies than the ARM7 family.
- The extra stages improve the overall performance of the processor. The memory system has been redesigned to follow the Harvard architecture, which separates the data D and instruction I buses.
- The next processors in the ARM9 family were based on the ARM9E-S core. This core is a synthesizable version of the ARM9 core with the E extensions.
- There are two variations: the ARM946E-S and the ARM966E-S.
- Both execute architecture v5TE instructions.
- They also support the optional embedded trace macrocell (ETM), which allows a developer to trace instruction and data execution in real time on the processor.
- This is important when debugging applications with time-critical segments.
- The latest core in the ARM9 product line is the ARM926EJ-S synthesizable processor core, announced in 2000.
- It is designed for use in small portable Java-enabled devices such as 3G phones and personal digital assistants (PDAs).

ARM10 FAMILY:

- The ARM10, announced in 1999, was designed for performance. It extends the ARM9 pipeline to six stages.
- It also supports an optional vector floating-point (VFP) unit, which adds a seventh stage to the ARM10 pipeline.
- The ARM1020E is the first processor to use an ARM10E core. Like the ARM9E, it includes the enhanced E instructions.

- ARM1026EJ-S is very similar to the ARM926EJ-S but with both MPU and MMU.
- This processor has the performance of the ARM10 with the flexibility of an ARM926EJ-S.

ARM11 FAMILY:

- The ARM1136J-S, announced in 2003, was designed for high performance and power efficient applications.
- ARM1136J-S was the first processor implementation to execute architecture ARMv6 instructions.
- It incorporates an eight-stage pipeline with separate loadstore and arithmetic pipelines.
- Included in the ARMv6 instructions are single instruction multiple data (SIMD) extensions for media processing, specifically designed to increase video processing performance.
- The ARM1136JF-S is an ARM1136J-S with the addition of the vector floating-point unit for fast floating-point operations.

SPECIALIZED PROCESSORS:

- StrongARM was originally co-developed by Digital Semiconductor and is now exclusively licensed by Intel Corporation.
- It has been popular for PDAs and applications that require performance with low power consumption.
- StrongARM was the first high-performance ARM processor to include a five-stage pipeline, but it does not support the Thumb instruction set.
- Intel's XScale is a follow-on product to the StrongARM and offers dramatic increases in performance.
- It is a Harvard architecture and is similar to the StrongARM, as it also includes an MMU.

This core is small and has low voltage and current requirements, which makes it attractive for smart card applications.

Comparison of RISC and CISC Processor:

CISC

A complex instruction set computer is a computer where single instructions can execute several low-level operations (such as a load from memory, an arithmetic operation, and a memory store) or are capable of multi-step operations or addressing modes within single instructions, as its name suggest "COMPLEX INSTRUCTION SET".

RISC

A reduced instruction set computer is a computer which only use simple instructions that can be divide into multiple instructions which perform low-level operation within single clock cycle, as its name suggest "REDUCED INSTRUCTION SET"

The CISC Approach :- The primary goal of CISC architecture is to complete a task in as few lines of assembly as possible. This is achieved by building processor hardware that is capable of understanding & executing a series of operations, this is where our CISC architecture introduced .

For this particular task, a CISC processor would come prepared with a specific instruction

When executed, this instruction

1. Loads the two values into separate registers
2. Multiplies the operands in the execution unit

3. And finally third, stores the product in the appropriate register.

Thus, the entire task of multiplying two numbers can be completed with one instruction:

MULT A,B <<<=====this is assembly statement

MULT is what is known as a “complex instruction.” It operates directly on the computer’s memory banks and does not require the programmer to explicitly call any loading or storing functions.

Advantage:-

1. Compiler has to do very little work to translate a high-level language statement into assembly
2. Length of the code is relatively short
3. Very little RAM is required to store instructions
4. The emphasis is put on building complex instructions directly into the hardware.

The RISC Approach :- RISC processors only use simple instructions that can be executed within one clock cycle.

1. “LOAD” which moves data from the memory bank to a register
2. “PROD” which finds the product of two operands located within the registers
3. “STORE” which moves data from a register to the memory banks.

In order to perform the exact series of steps described in the CISC approach, a programmer would need to code four lines of assembly:

LOAD R1, A <<<=====this is assembly statement

LOAD R2,B <<<=====this is assembly statement

PROD A, B <<<=====this is assembly statement

STORE R3, A <<<=====this is assembly statement

At first, this may seem like a much less efficient way of completing the operation. Because there are more lines of code, more RAM is needed to store the assembly level instructions. The compiler must also perform more work to convert a high-level language statement into code of this form.

Advantage:-

1. Each instruction requires only one clock cycle to execute, the entire program will execute in approximately the same amount of time as the multi-cycle “MULT” command.
2. These RISC “reduced instructions” require less transistors of hardware space than the complex instructions, leaving more room for general purpose registers. Because all of the instructions execute in a uniform amount of time (i.e. one clock)
3. Pipelining is possible.

LOAD/STORE mechanism:- Separating the “LOAD” and “STORE” instructions actually reduces the amount of work that the computer must perform. After a CISC-style “MULT” command is executed, the processor automatically erases the registers. If one of the operands needs to be used for another computation, the processor must re-load the data from the memory bank into a register. In RISC, the operand will remain in the register until another value is loaded in its place.

CISC

RISC

Emphasis on hardware	Emphasis on software
Includes multi-clock	Single-clock
complex instructions	reduced instruction only
Memory-to-memory: "LOAD" and "STORE" incorporated in instructions	Register to register: "LOAD" and "STORE" are independent instructions
high cycles per second, Small code sizes	Low cycles per second, large code sizes
Transistors used for storing complex instructions	Spends more transistors on memory registers

Comparison of ARM 9 five stage and ARM 10 six stage pipeline

The 5-stage ARM pipeline

- Fetch
- Decode
 - instruction decode and register read
- Execute
 - shift and ALU
- Memory
 - data memory access
- Write-back

Reducing the CPI

- ARM7 uses the memory on nearly every clock cycle
 - for either instruction fetch or data transfer
- therefore a reduced CPI requires more than one memory access per clock cycle

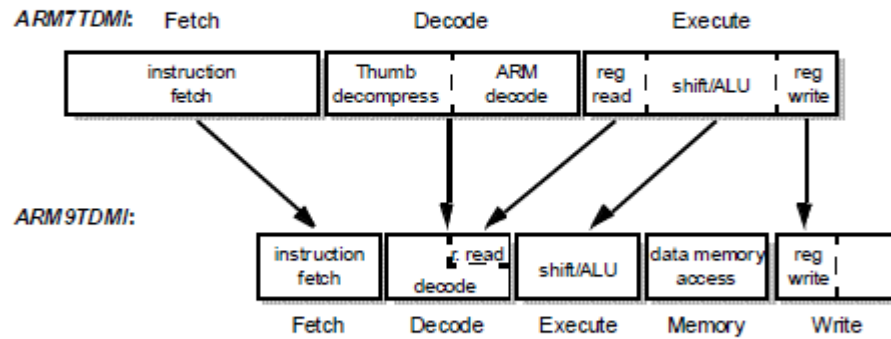
Possible solutions are:

- separate instruction and data memories
- double-bandwidth memory (e.g. ARM8)

ARM9TDMI

- a 'classic' Harvard architecture 5-stage pipeline
 - separate instruction and data memory ports
- with full support for Thumb and EmbeddedICE debug
- aimed at significantly higher performance than the ARM7TDMI
 - enhanced pipeline operates at 100-200 MHz

ARM9TDMI pipeline



- Thumb instructions are decoded directly

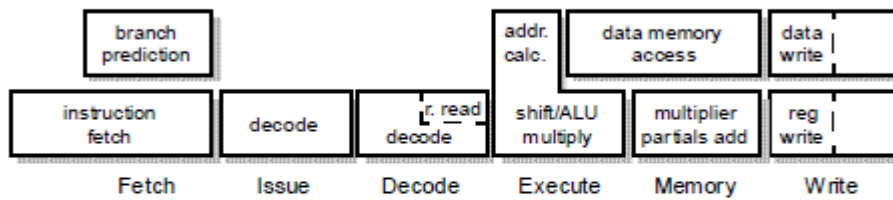
ARM10TDMI

- aimed at significantly higher performance than the ARM9TDMI
- achieved through use of:
 - higher clock rate
 - 64-bit I- and D-memory buses
 - branch prediction
 - hit-under-miss D-memory interface

The 6-stage ARM pipeline

- Fetch
- Issue
- Decode
 - instruction decode and register read
- Execute
 - shift and ALU
- Memory
 - data memory access
- Write-back

ARM10TDMI pipeline



- Additional time allowed for
 - I- and D-memory accesses
 - instruction decode
- 6-stage pipeline

2 MARKS

1. What is an embedded system? (Nov 2016) (Nov 2017)

An embedded system employs a combination of hardware & software (a “computational engine”) to perform a specific function; is part of a larger system that may not be a “computer”; works in a reactive and time-constrained environment.

2. What are the components of the Embedded Systems?

An embedded system is basically a computer controlled device designed to perform some specific tasks. In most cases these tasks revolve around real-time control of machines or processes. Embedded systems are more cost effective to implement than systems composed of general purpose computers, such as PCs. The components of ES are,

- Memory
- System Clock
- Peripherals

3. What are the characteristics of an embedded system?

The typical characteristics of the embedded Systems are as follows:

1) Embedded systems are designed to do some specific task, rather than be a general-purpose computer for multiple tasks. Some also have real-time performance constraints that must be met, for reason such as safety and usability; others may have low or no performance requirements, allowing the system hardware to be simplified to reduce costs.

2) Embedded systems are not always separate devices. Most often they are physically built-in to the devices they control.

3) The software written for embedded systems is often called firmware, and is stored in read-only memory or Flash memory chips rather than a disk drive. It often runs with limited computer hardware resources: small or no keyboard, screen, and little memory.

4. What are the advantages of embedded system?

The advantages of the embedded system are Customization yields lower area, power, cost.

5. What are the disadvantages of embedded system?

- Higher HW/software development overhead
- design, compilers, debuggers, ...
- May result in delayed time to market!

6. What is a Microprocessor?

A silicon chip that contains a CPU. In the world of personal computers, the terms microprocessor and CPU are used interchangeably. At the heart of all personal computers and most workstations sits a microprocessor. Microprocessors also control the logic of almost all digital devices, from clock radios to fuel-injection systems for automobiles.

7. What is a Microcontroller?

A **microcontroller** is a small and low-cost computer built for the purpose of dealing with specific tasks, such as displaying information in a microwave LED or receiving information from a television's remote control. **Microcontrollers** are mainly used in products that require a degree of control to be exerted by the user.

8. What are differences between Microprocessor and Microcontroller?

MICROPROCESSOR	MICROCONTROLLERS
The functional blocks are ALU, registers, timing & control unit.	It includes functional blocks of microprocessors & in addition has timer, parallel i/o, RAM, EPROM, and ADC & DAC.
Bit handling instruction is less, one or two type only.	Many types of bit handling instruction.
Rapid movements of code and data between external memory & MP.	Rapid movements of code and data within me.
It is used for designing general purpose digital computers system.	They are used for designing application specific dedicated systems.

9. What are the various embedded system designs?

- Modeling
- Refining (or “partitioning”)
- HW-SW partitioning

10. List the various processors that are present?

- General Purpose Processor (GPP)

- Microprocessor
- Microcontroller
- Embedded Processor
- Digital Signal Processor
- Application Specific System Processor (ASSP)
- Multi-Processor System using GPPS

11. What is the Embedded Processor?

Special microprocessor and microcontrollers often called Embedded Processor. An embedded processor is used when fast processing fast context – switching and atomic ALU operations are needed.

Examples: ARM7, INTEL 1960, AMD 29050.

12. List the applications of Embedded Systems? (May 2016)

Embedded Systems: Applications:

- Consumer electronics, e.g., cameras, camcorders,
- Consumer products, e.g., washers, microwave ovens,
- Automobiles (anti-lock braking, engine control,)
- Industrial process controllers & avionics/defense applications
- Computer/Communication products, e.g., printers, FAX machines,
- Emerging multimedia applications & consumer electronics

13. What are the hardware units present in the embedded system?

- A Microprocessor
- A Large Memory (Ram, ROM and caches)
- Input Units (Keyboard, Mouse, Scanner, etc)
- Output Units (Monitor, Printer, etc)
- Networking Units (Ethernet Card, Drivers, etc)
- I/O Units (Modem, Fax cum modem, etc)

14. What is embedded software? (May 2017)

An embedded system needs software to drive it. The software components within an embedded system often encompass the technology that adds value to the system and defines what it does and how well it does it.

15. Define the term firmware?

The software components can run from ROM or RAM. ROM code that is fixed on the device (for example, the initialization code) is called firmware.

16. What is ARM?

ARM, originally **Acorn RISC Machine**, is a family of reduced instruction set computing (RISC) architectures for computer processors, configured for various environments

17. What is ARM processor?

The ARM processor core is a key component of many successful 32-bit embedded systems. ARM cores are widely used in mobile phones, handheld organizers, and a multitude of other everyday portable consumer devices.

17. What is a RISC machine?

Reduced instruction set computing, or **RISC**, is a CPU design strategy based on the insight that a simplified instruction set (as opposed to a complex set) provides higher performance when combined with a microprocessor architecture capable of executing those instructions using fewer microprocessor cycles per instruction. RISC design places greater demands on the compiler.

18. What is a CISC machine?

Complex instruction set computing (CISC) is a processor design where single instructions can execute several low-level operations (such as a load from memory, an arithmetic operation, and a memory store) or are capable of multi-step operations or addressing modes within single instructions. (CISC) relies more on the hardware for instruction functionality

19. List the software components of an embedded system

- Initialization code
- Operating systems
- Device drivers
- Application

20. What is the use of a controller?

There is a special type of peripheral called a controller, which embedded systems use to configure higher-level functions such as memory and interrupts.

21. What is load-store architecture?

It has two instruction types for transferring data in and out of the processor: load instructions copy data from memory to registers in the core, and conversely the store instructions copy data from registers to memory. There are no data processing instructions that directly manipulate data in memory.

22. What is a register file?

Data processing is carried out in registers. Data items are placed in the register file—a storage bank made up of 32-bit registers.

23. List the registers of ARM

There are up to 18 active registers: 16 data registers and 2 processor status registers. The data registers are visible to the programmer as r0 to r15. The ARM processor has three registers assigned to a particular task or special function: r13, r14, and r15. They are frequently given different labels to differentiate them from the other registers. All the registers shown are 32 bits in size.

24. What is stack pointer?

Register r13 is traditionally used as the stack pointer (sp) and stores the head of the stack in the current processor mode.

25. What is a link register in ARM?

Register r14 is called the link register (lr) and is where the core puts the return address whenever it calls a subroutine.

26. What does program counter contain?

Program counter contains the address of the next instruction to be fetched by the processor. In ARM, Register r15 is the program counter (pc)

27. Can the special registers of ARM used as general purpose registers?

R13 and r14 can also be used as general-purpose registers, which can be particularly useful since these registers are banked during a processor mode change. However, r13 cannot be used as a general register when the processor is running any form of operating system because operating systems often assume that r13 always points to a valid stack frame.

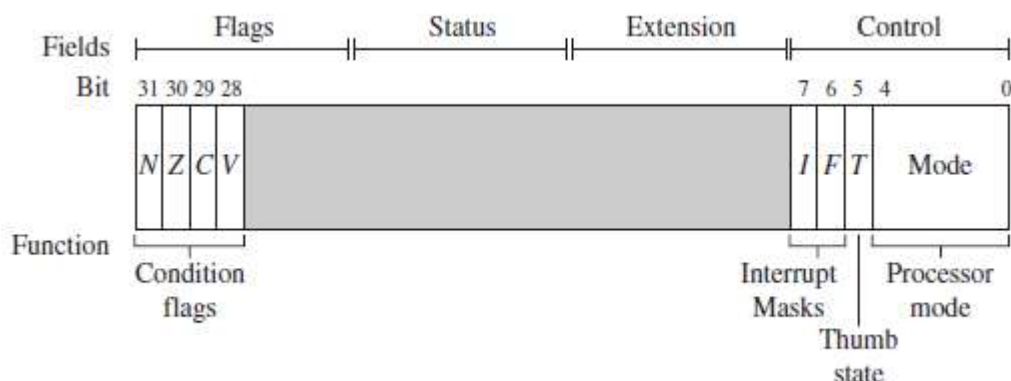
28. List the status registers of ARM.

There are two program status registers: cpsr and spsr (the current and saved program status registers, respectively).

29. What is the use of CPSR? (Nov 2016)

The ARM core uses the cpsr to monitor and control internal operations. The cpsr is a dedicated 32-bit register and resides in the register file.

30. Draw the layout of a program status register. (Nov 2017)



31. List the modes of ARM processor.

There are seven processor modes in total: six privileged modes (abort, fast interrupt request, interrupt request, supervisor, system, and undefined) and one nonprivileged mode (user).

32. What is a privileged mode & non-privileged mode?

Each processor mode is either privileged or nonprivileged: A privileged mode allows full read-write access to the cpsr. Conversely, a nonprivileged mode only allows read access to the control field in the cpsr but still allows read-write access to the condition flags. six privileged modes (abort, fast interrupt request, interrupt request, supervisor, system, and undefined) and one nonprivileged mode (user).

33. What are banked registers?

A complete ARM register set contains 37 registers in the register file. Of those, 20 registers are hidden from a program at different times. These registers are called banked registers. They are available only when the processor is in a particular mode

34. What are exceptions and interrupts?

Exceptions and interrupts suspend the normal execution of sequential instructions and jump to a specific location. The following exceptions and interrupts cause a mode change: reset, interrupt request, and fast interrupt request, software interrupt, data abort, prefetch abort, and undefined instruction.

35. In what mode does the processor starts when powered on?

When power is applied to the core, it starts in supervisor mode, which is privileged. Starting in a privileged mode is useful since initialization code can use full access to the cpsr to set up the stacks for each of the other modes.

36. List the instruction sets of ARM processor.

The state of the core determines which instruction set is being executed. There are three instruction sets: ARM, Thumb, and Jazelle. The ARM instruction set is only active when the processor is in ARM state. Similarly the Thumb instruction set is only active when the processor is in Thumb state.

37. What is Jazelle?

A third instruction set of ARM is called Jazelle. Jazelle executes 8-bit instructions and is a hybrid mix of software and hardware designed to speed up the execution of Java byte codes.

38. What are interrupt masks?

Interrupt masks are used to stop specific interrupt requests from interrupting the processor. The cpsr has two interrupt mask bits, 7 and 6 (or I and F), which control the masking of IRQ and FIQ, respectively.

39. List the condition flags.

Flag	Flag name	Set when
Q	Saturation	the result causes an overflow and/or saturation
V	oVerflow	the result causes a signed overflow
C	Carry	the result causes an unsigned carry
Z	Zero	the result is zero, frequently used to indicate equality
N	Negative	bit 31 of the result is a binary 1

40. What are the advantages of using pipeline?

A pipeline is the mechanism a RISC processor uses to execute instructions. Using a pipeline speeds up execution by fetching the next instruction while other instructions are being decoded and executed.

41. List the pipeline length of various ARM processors

ARM7 uses a 3-stage pipeline: Fetch – Decode – Execute

ARM9 uses a 5-stage pipeline: Fetch – Decode – Execute – Memory – Write

ARM10 uses a 6-stage pipeline: Fetch – Issue - Decode – Execute – Memory – Write

42. Define the term vector table (May 2017)

When an exception or interrupt occurs, the processor sets the pc to a specific memory address. The address is within a special address range called the vector table. The entries in the vector table are instructions that branch to specific routines designed to handle a particular exception or interrupt.

43. Sketch the nomenclature ARM uses?

ARM{x}{y}{z}{T}{D}{M}{I}{E}{J}{F}{-S}

x—family

y—memory management/protection unit

z—cache

T—Thumb 16-bit decoder

D—JTAG debug

M—fast multiplier

I—EmbeddedICE macrocell

E—enhanced instructions (assumes TDMI)

J—Jazelle

F—vector floating-point unit

S—synthesizable version

ARM nomenclature.

The letters and numbers after the word “ARM” indicate the features a processor may have.

44. What is embedded trace macrocell (ETM)?

Embedded trace macrocell (ETM), allows a developer to trace instruction and data execution in real time on the processor. This is important when debugging applications with time-critical segments. ARM9 family also support the optional embedded trace macrocell (ETM)

45. Name the processor that uses Jazelle technology.

ARM926EJ-S is the first ARM processor core to include the Jazelle technology, which accelerates Java byte code execution. It features an MMU, configurable TCMs, and D +I caches with zero or nonzero wait state memories.

46. What is Strong ARM?

Strong ARM was the first high-performance ARM processor to include a five-stage pipeline, but it does not support the Thumb instruction set. It has been popular for PDAs and applications that require performance with low power consumption. It is Harvard architecture

47. What are the core extensions of ARM?

The core extensions include the following: Caches are used to improve the overall system performance. TCMs are used to improve deterministic real-time response Memory management is used to organize memory and protect system resources.

Coprocessors are used to extend the instruction set and functionality. Coprocessor 15 controls the cache, TCMs, and memory management.

48. Is LCD Projector an Embedded system? Justify. (May 2016)

Yes . LCD Projector is an embedded system. It does a single functional task.